



Πανεπιστήμιο Πειραιώς

Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών  
Τμήμα Ψηφιακών Συστημάτων  
Προπτυχιακό Πρόγραμμα Σπουδών

**Συστήματα Ευφύων Πρακτόρων**

Καθηγητής: Γεώργιος Βούρος

*Βοηθητικό Προσωπικό: Ναταλία Κολιού, Χρήστος Κυριαζόπουλος*

*1<sup>η</sup> Ομαδική Εργασία (2023-2024)*

Alexander Sviridov

[alex.sviridov123@gmail.com](mailto:alex.sviridov123@gmail.com)

E14166

Πειραιάς

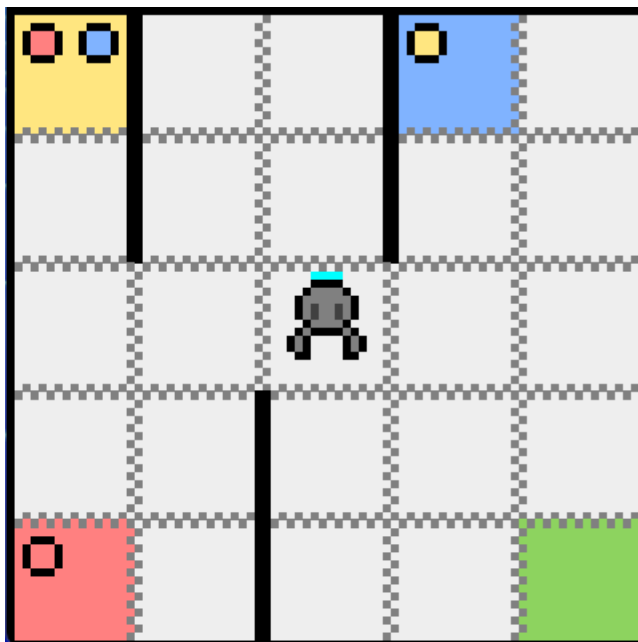
31/1/2024

## Περιεχόμενα

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Περιγραφή.....                                                           | 3  |
| Το Περιβάλλον .....                                                      | 4  |
| Το Μοντέλο του Περιβάλλοντος ( <i>Model</i> ) .....                      | 4  |
| Το Κελί ( <i>Cell</i> ) .....                                            | 5  |
| Το Ρομπότ ( <i>Robot</i> ).....                                          | 6  |
| Το Περιβάλλον ως Οντότητα ( <i>Env</i> ) .....                           | 8  |
| Ο Πράκτορας.....                                                         | 9  |
| Οι Πεποιθήσεις του Πράκτορα.....                                         | 9  |
| Οι Στόχοι του Πράκτορα.....                                              | 10 |
| Τα Σχέδια του Πράκτορα .....                                             | 11 |
| Ο Σχεδιαστής Πλάνου του Πράκτορα.....                                    | 16 |
| Το Πρόβλημα και ο Αλγόριθμος της Καθοδήγησης ( <i>Navigation</i> ) ..... | 16 |
| Το Πρόβλημα και ο Αλγόριθμος της Δρομολόγησης ( <i>Router</i> ).....     | 18 |
| Πειραματικά Αποτελέσματα & Συμπεράσματα .....                            | 23 |
| Εκτέλεση της Προσομοίωσης .....                                          | 25 |
| Βιβλιογραφία.....                                                        | 28 |

## Περιγραφή

Η επισυναπτόμενη εικόνα είναι το  $5 \times 5$  πλέγμα κελιών του κόσμου του **πράκτορα** ρομπότ (taxi στην αρχική εκφώνηση).



Υπάρχουν τέσσερα (4) χρωματιστά κελιά στον κόσμο αυτόν:

1. το κόκκινο (*red*),
2. το πράσινο (*green*),
3. το μπλε (*blue*) και
4. το κίτρινο (*yellow*).

Σε κάθε επεισόδιο, ο πράκτορας ξεκινά σε ένα τυχαία επιλεγμένο κελί του πλέγματος. Κάθε πελάτης, εμφανίζεται σε τυχαίο χρωματιστό κελί (αφετηρία) με προορισμό ένα άλλο τυχαία επιλεγμένο χρωματιστό κελί. Ο πράκτορας πρέπει να μεταβεί στην αφετηρία του πελάτη, να τον «φορτώσει» και να πάει στον προορισμό του, όπου και «εκφορτώνει» τον πελάτη. (Θεωρήστε ότι ο πράκτορας «φορτώνει» και «εκφορτώνει» τον πελάτη ακόμα και στην περίπτωση που η αρχική και τελική του θέση είναι η ίδια.) **Πολλαπλοί πελάτες εμφανίζονται στις χρωματιστές θέσεις τυχαία.**

Θεωρήστε ότι το κάθε κελί μπορεί να περιέχει μέχρι ένα συγκεκριμένο πλήθος πελατών *maxNumCellClients*. Το επεισόδιο ολοκληρώνεται όταν ο πράκτορας πραγματοποιήσει ένα μέγιστο πλήθος δράσεων *maxActions*.

Ο πράκτορας ενημερώνεται από το περιβάλλον για την αφετηρία και τον προορισμού κάθε νεοεμφανιζόμενου πελάτη.

Ο πράκτορας δεν λαμβάνει καμία άμεση αντίληψη από το περιβάλλον για τους τοίχους, ωστόσο αντιλαμβάνεται τις συντεταγμένες του τρέχοντος κελιού στο οποίο βρίσκεται.

Ο πράκτορας έχει την ικανότητα να πραγματοποιεί τις ακόλουθες δράσεις, με κόστος  $-1$  για κάθε την κάθε δράση:

0. Μετακίνηση στο γειτονικό του κελί προς τα δεξιά (*right*)
1. Μετακίνηση στο γειτονικό του κελί προς τα πάνω (*up*)
2. Μετακίνηση στο γειτονικό του κελί προς τα αριστερά (*left*)
3. Μετακίνηση στο γειτονικό του κελί προς τα κάτω (*down*)
4. Εκφόρτωση πελάτη στο τρέχον κελί του
5. Φόρτωση πελάτη από το τρέχον κελί του με προορισμό το κόκκινο (*red*) κελί
6. Φόρτωση πελάτη από το τρέχον κελί του με προορισμό το πράσινο (*green*) κελί
7. Φόρτωση πελάτη από το τρέχον κελί του με προορισμό το μπλε (*blue*) κελί
8. Φόρτωση πελάτη με προορισμό το κίτρινο (*yellow*) κελί

Ο πράκτορας λαμβάνει ανταμοιβή  $+20$  κάθε φορά που «εκφορτώνει» κάποιον πελάτη στο κελί προορισμού του, ενώ έχει κόστος  $-10$  αν «εκφορτώσει» κάποιον πελάτη σε λάθος κελί. Επιπλέον, το κόστος στη περίπτωση μιας σύγκρουσης του ταξί με κάποιον τοίχο είναι  $-100$ .

## Το Περιβάλλον

### Το Μοντέλο του Περιβάλλοντος (*Model*)

Το μοντέλο του περιβάλλοντος ορίζει ως την βασική υποδομή του ένα  $5 \times 5$  πλέγμα κελιών (*GRID*).

Ορίζονται τέσσερις κατευθύνσεις μετακίνησης επί του πλέγματος, βάσει του παρακάτω πίνακα κατευθυντικών ολισθήσεων (*Directional Shifts, DS*)

| Κατεύθυνση, $d$                | Δεξιά, 0 | Πάνω, 1   | Αριστερά, 2 | Κάτω, 3  |
|--------------------------------|----------|-----------|-------------|----------|
| Κατευθυντική Ολίσθηση, $DS[d]$ | $[0, 1]$ | $[-1, 0]$ | $[0, -1]$   | $[1, 0]$ |

Έστω το κελί του πλέγματος  $GRID[row][col]$ , τότε οι συντεταγμένες του γειτονικού αυτού,  $row'$  και  $col'$ , προς την κατεύθυνση  $d$  βάσει του  $DS$ , μπορούν να υπολογιστούν ως εξής:

$$\begin{aligned} row' &\leftarrow row + DS[d][0] \\ col' &\leftarrow col + DS[d][1] \end{aligned}$$

και το γειτονικό κελί του  $GRID[row][col]$ , προς την κατεύθυνση  $d$ , θα είναι το  $GRID[row'][col']$ .

Ορίζονται επίσης τέσσερα χρωματιστά κελιά στο πλέγμα, με συντεταγμένες όπως αυτές δίνονται στον παρακάτω πίνακα:

| Χρώμα, $c$ | Κανένα, 0 | Κόκκινο, 1 | Πράσινο, 2 | Μπλε, 3 | Κίτρινο, 4 |
|------------|-----------|------------|------------|---------|------------|
|------------|-----------|------------|------------|---------|------------|

|                                                                    |             |        |        |        |        |
|--------------------------------------------------------------------|-------------|--------|--------|--------|--------|
| Συντεταγμένες<br>πλέγματος<br>χρωματιστού<br>κελιού,<br>[row, col] | <i>null</i> | [4, 0] | [4, 4] | [0, 3] | [0, 0] |
|--------------------------------------------------------------------|-------------|--------|--------|--------|--------|

Το μοντέλο του περιβάλλοντος ορίζει την δοθείσα πολιτική αξιολόγησης *evaluation* για τον πράκτορα. Η πολιτική αυτή βασίζεται σε συγκεκριμένα γεγονότα. Κάθε γεγονός αντιστοιχίζεται με μια τιμή κόστους ή αμοιβής. Κάθε φορά που συμβαίνει ένα από τα γεγονότα αυτά, η αντίστοιχη τιμή κόστους ή αμοιβής αποδίδεται (προστίθεται) στο τρέχον *evaluation*. Τα γεγονότα αυτά και οι αντίστοιχες τιμές δίνονται στον ακόλουθο πίνακα:

| Γεγονός                 | Περιγραφή                                                                     | Τιμή κόστους/αμοιβής |
|-------------------------|-------------------------------------------------------------------------------|----------------------|
| <b><i>COLLISION</i></b> | Σύγκρουση ρομπότ με τοίχο.                                                    | −100                 |
| <b><i>PENALTY</i></b>   | Εκφόρτωση πελάτη από το ρομπότ σε κελί που δεν είναι ο προορισμός του πελάτη. | −10                  |
| <b><i>ACTION</i></b>    | Πραγματοποίηση μιας οποιασδήποτε δράσης από το ρομπότ.                        | −1                   |
| <b><i>REWARD</i></b>    | Εκφόρτωση πελάτη από το ρομπότ σε κελί που είναι ο προορισμός του πελάτη.     | +20                  |

Η αριθμητικά υψηλότερη αξιολόγηση θεωρείται και καλύτερη.

### Το Κελί (*Cell*)

Το κάθε κελί  $C$  μπορεί να προσδιοριστεί μοναδικά μέσω της γραμμής ( $C.row$ ) και της στήλης ( $C.col$ ) πλέγματος.

Κάθε κελί,  $C$ , μπορεί να παρουσιάζει τοίχο κατά μήκος οποιασδήποτε πλευράς του, προς οποιαδήποτε κατεύθυνση  $d$ , στην οποία περίπτωση, η μετακίνηση του ρομπότ από το κελί  $C$  στο γειτονικό κελί στην κατεύθυνση  $d$  είναι αδύνατη και η κατεύθυνση μετακίνησης  $d$  από το κελί  $C$  αναφέρεται ως μπλοκαρισμένη και θα ισχύει η συνθήκη  $\neg(C.isDirectionFree[d])$ .

Αντίθετα, εάν δεν υπάρχει τοίχος στην εν λόγω κατεύθυνση, η κατεύθυνση  $d$  του κελιού  $C$  αναφέρεται ως ελεύθερη (ως προς τη μετακίνηση) και θα ισχύει η συνθήκη  $C.isDirectionFree[d]$ .

Η δομή *isDirectionFree* αρχικοποιείται κατάλληλα για κάθε κελί κατά την αρχή της προσομοίωσης.

Κάθε κελί  $C$  χαρακτηρίζεται από ένα χρώμα,  $C.color$ . Η προκαθορισμένη τιμή χρώματος για κάθε κελί είναι το 0 (άχρωμο). Σε τέσσερα από τα κελιά του πλέγματος, στα οποία θα αναφερόμαστε ως χρωματιστά, ανατίθενται με ένα-προς-ένα αντιστοιχία τα χρώματα 1, 2, 3, και 4.

Τα χρωματιστά κελιά επιλέγονται ως πηγές ή αφετηρίες πελατών.

Ως πελάτης θεωρείται ένα αντικείμενο το οποίο εμφανίζεται σε χρωματιστό κελί, το οποίο αποτελεί την αφετηρία του πελάτη, και έχει ως προορισμό του ένα άλλο χρωματιστό κελί. Κάθε πελάτης μπορεί να αναπαρασταθεί με το χρώμα του κελιού προορισμού του,  $c$ .

Το κάθε κελί  $C$  διατηρεί ένα σύνολο ή λίστα πελατών,  $C.clients$ , η οποία έχει μια μέγιστη χωρητικότητα πελατών,  $MAX\_NUM\_CELL\_CLIENTS = 4$ .

Κάθε κελί  $C$  παρέχει διεπαφές ή μεθόδους, μέσω των οποίων άλλες οντότητες μπορούν να αλληλοεπιδρούν με αυτό. Αυτές οι διεπαφές-μέθοδοι είναι:

➤  $C.spawn()$

Αυτή η μέθοδος εμφανίζει, κατά το 25% των κλήσεων της, έναν νέο τυχαίο πελάτη  $c$ , με αφετηρία το κελί  $C$ .

Στην περίπτωση όπου το κελί  $C$  δεν μπορεί να χωρέσει άλλον πελάτη, δηλαδή  $MAX\_NUM\_CELL\_CLIENTS \leq C.clients.size()$ , τότε προφανώς η μέθοδος αυτή δεν θα εμφανίσει νέο πελάτη.

➤  $C.extract(c)$

Αυτή η μέθοδος εξάγει τον πελάτη  $c$  από το κελί  $C$ , διαγράφοντας τον πελάτη  $c$  από την λίστα των πελατών του κελιού,  $C.clients$ , και επιστρέφοντας τον πελάτη.

Εάν δεν βρισκόταν κανένας πελάτης  $c$  στο κελί  $C$  κατά την κλήση της μεθόδου αυτής, τότε δεν επιστρέφει πελάτη, που υποδεικνύεται από την τιμή επιστροφής 0.

➤  $C.insert(c)$

Αυτή η μέθοδος εισάγει τον πελάτη  $c$  στο κελί  $C$ .

Εάν το χρώμα προορισμού του πελάτη,  $c$ , ταυτίζεται με το χρώμα του κελιού, τότε προστίθεται στο τρέχον *evaluation* η τιμή αμοιβής εξυπηρέτησης,  $REWARD = +20$ . Σε αυτήν την περίπτωση, ο πελάτης «εξέρχεται» του περιβάλλοντος επιτυχώς.

Αλλιώς, πρόκειται για εσφαλμένη εξυπηρέτηση και η τιμή πέναλτι,  $PENALTY = -10$ , αποδίδεται στο τρέχον *evaluation*. Εάν το κελί χωρά τον εισαγόμενο πελάτη, τότε τον προσθέτει στην λίστα πελατών  $C.clients$ . Σε αντίθετη περίπτωση, ο πελάτης «χάνεται».

## Το Ρομπότ (Robot)

Σε οποιοδήποτε στιγμιότυπο εκτέλεσης, το ρομπότ μπορεί να κρατάει το πολύ έναν πελάτη. Αρχικά δεν κουβαλάει κανέναν πελάτη,  $robot.carry \leftarrow 0$ .

Το ρομπότ (*robot*) σε κάθε στιγμιότυπο εκτέλεσης μπορεί να βρίσκεται σε ένα οποιοδήποτε κελί του πλέγματος, *robot.cell* (στο οποίο, κατά την υλοποίηση του συστήματος, διατηρεί μια αναφορά).

Το ρομπότ διαθέτει μια ποσότητα μονάδων ενέργειας, *robot.energy*, αρχικά ίση με *MAX\_ROBOT\_ENERGY*.

Το ρομπότ, όπως περιγράφεται ακολούθως, έχει την δυνατότητα να εκτελεί διάφορες δράσεις. Προκειμένου το ρομπότ να επιχειρήσει να πραγματοποιήσει μια οποιαδήποτε δράση, απαιτείται η κατανάλωση μιας μονάδας ενέργειας. Όταν η ενέργεια του ρομπότ τελειώσει, το ρομπότ δεν θα μπορεί να πραγματοποιήσει άλλη δράση.

Το ρομπότ διαθέτει το ακόλουθο σύνολο δράσεων ή μεθόδων που του επιτρέπει να αλληλοεπιδράει με άλλες οντότητες του περιβάλλοντος. Η πραγματοποίηση οποιασδήποτε εκ των δράσεων αυτών απαιτεί την κατανάλωση μιας μονάδας ενέργειας και η εκτέλεση οποιασδήποτε από τις δράσεις αποδίδει το κόστος δράσης  $ACTION := -1$  στο τρέχον *evaluation*. Πέρα αυτών, για την κάθε μεμονωμένη δράση ισχύει:

➤ *robot.move(d)*

Με αυτήν του τη δράση το ρομπότ επιχειρεί να μετακινηθεί από το τρέχον κελί του, *robot.cell* στο γειτονικό κελί αυτού προς την κατεύθυνση *d*.

Στην περίπτωση που ισχύει η συνθήκη  $\neg(robot.cell.isDirectionFree[d])$ , προκύπτει γεγονός σύγκρουσης του ρομπότ με τοίχο, με αποτέλεσμα το ρομπότ να παραμείνει στο τρέχον κελί, καθώς και η απόδοση του κόστους σύγκρουσης  $COLLISION := -100$  στο τρέχον *evaluation*.

Αν από την άλλη ισχύει η συνθήκη *robot.cell.isDirectionFree[d]*, τότε το ρομπότ μετακινείται επιτυχώς στο γειτονικό κελί αυτού προς την κατεύθυνση *d*.

➤ *robot.load(c)*

Με αυτήν τη δράση, το ρομπότ λαμβάνει τον πελάτη *c* από το τρέχον κελί του, με κλήση της μεθόδου του κελιού *robot.cell.extract(c)* και ανατίθεται το αποτέλεσμα της μεθόδου ως τον πελάτη που πλέον κρατά το ρομπότ, δηλαδή,  $robot.carry \leftarrow robot.cell.extract(c)$ .

Βέβαια, εάν το ρομπότ ήδη κρατούσε πελάτη, τότε η δράση αυτή αποτυγχάνει.

➤ *robot.unload( )*

Με αυτή τη δράση το ρομπότ εκφορτώνει τον πελάτη που κρατά στο τρέχον κελί του, με κλήση της μεθόδου `robot.cell.insert(robot.carry)`, και βέβαια κατόπιν της εκφόρτωσης το ρομπότ δεν θα κρατάει πελάτη, δηλαδή  $c \leftarrow 0$ .

## Το Περιβάλλον ως Οντότητα (*Env*)

Το περιβάλλον, *Env*, υλοποιείται ως αντιδραστικός πράκτορας, που αλληλοεπιδρά με τον πράκτορα. Σε κάθε κύκλο αλληλοεπίδρασης, το περιβάλλον, *Env*, ενημερώνει τον πράκτορα με αντιλήψεις σχετικά με τη τρέχουσα κατάσταση του μοντέλου του περιβάλλοντος. Κατόπιν της λήψης του νέου συνόλου αντιλήψεων από το *Env*, ο πράκτορας ανταποκρίνεται με μια δράση. Το *Env* εφαρμόζει τη δράση αυτή επί του μοντέλου, το οποίο ενδεχομένως τροποποιείται. Έτσι ολοκληρώνεται ένας κύκλος και ξεκινάει ο νέος κύκλος αλληλοεπίδρασης μεταξύ του περιβάλλοντος και του πράκτορα.

Στο σύστημα μας, το σύνολο των αντιλήψεων με το οποίο ενημερώνει το *Env* τον πράκτορα είναι:

❖ *energy(E)*

Όπου *E* η τρέχουσα ενέργεια του ρομπότ.

❖ *carry(C)*

Όπου *C* ο τρέχον πελάτης (ως το χρώμα προορισμού του πελάτη) που κρατάει το ρομπότ.

❖ *robot(RobotCell)*

Όπου  $RobotCell := [Row, Col]$  οι συντεταγμένες του τρέχοντος κελιού στο οποίο βρίσκεται το ρομπότ.

❖ *nodes(NDs = [RobotCell, RedCell, GreenCell, BlueCell, YellowCell])*

Όπου:

0.  $RobotCell := [RobotRow, RobotCol]$  οι συντεταγμένες του κελιού του ρομπότ.
1.  $RedCell := [RedRow, RedCol]$  οι συντεταγμένες του κόκκινου κελιού.
2.  $GreenCell := [GreenRow, GreenCol]$  οι συντεταγμένες του πράσινου κελιού.
3.  $BlueCell := [BlueRow, BlueCol]$  οι συντεταγμένες του μπλε κελιού.
4.  $YellowCell := [YellowRow, YellowCol]$  οι συντεταγμένες του κίτρινου κελιού.

❖ *route(R)*

Σε κάθε κύκλο αλληλοεπίδρασης το *Env* επικαλείται τη μέθοδο εμφάνισης νέου πελάτη του μοντέλου, `model.spawn()`. Εάν κατά αυτήν εμφανιστεί νέος πελάτης με χρώμα προορισμού *Dst* στο κελί χρώματος *Src*, τότε θα προστεθεί στο παραπάνω σύνολο

αντιλήψεων η αντίληψη  $route(R)$ , όπου  $R := [Src, Dst]$  το ζεύγος χρώματος αφετηρίας-προορισμού του νεοεμφανιζόμενου πελάτη.

Τα παραπάνω είναι το σύνολο αντιλήψεων επί του περιβάλλοντος που λαμβάνει ο πράκτορας σε κάθε κύκλο αλληλοεπίδρασης του με το περιβάλλον  $Env$ . Τα πιο πρόσφατα στιγμιότυπα των παραπάνω αντιλήψεων θα είναι και μέλη της τρέχουσας βάσης γνώσης του πράκτορα.

Οι δράσεις του πράκτορα τις οποίες αναγνωρίζει το  $Env$  και τις οποίες εφαρμόζει επί του μοντέλου είναι αυτές του ρομπότ. Όποτε λάβει μια εκ των δράσεων,  $move$ ,  $load$  ή  $unload$  θα επικαλεστεί την αντίστοιχη δράση επί του μοντέλου. Σε οποιαδήποτε άλλου τύπου δράσης το  $Env$  δεν προβαίνει σε εφαρμογή αυτής.

## Ο Πράκτορας

### Οι Πεποιθήσεις του Πράκτορα

Η βάση γνώσης του πράκτορα αποτελείται από τις πεποιθήσεις του πράκτορα, δηλαδή τί «πιστεύει» ο πράκτορας για την τρέχουσα κατάσταση του περιβάλλοντος. Αυτές οι πεποιθήσεις δεν είναι κατ' ανάγκη ορθές ή πλήρεις.

Πέρα των αντιλήψεων που παρέχει το περιβάλλον,  $Env$ , στον πράκτορα, των οποίων τα πιο πρόσφατα στιγμιότυπα είναι μέλη της βάσης γνώσης, ο πράκτορας διατηρεί τις ακόλουθες πεποιθήσεις:

#### ❖ $free\_cell\_directions(FCDs)$

Όπου  $FCDs$  λίστα (αρχικά άδεια). Η πεποίθηση αυτή χρησιμεύει ως εξής: όποτε ο πράκτορας διαπιστώσει ότι η μετακίνηση του ρομπότ από το κελί  $Cell := [Row, Col]$  στο γειτονικό αυτού προς την κατεύθυνση  $Dir$  είναι επιτυχής, τότε προσθέτει το στοιχείο  $[Cell, Dir]$  στην λίστα  $FCDs$  της πεποίθησης  $free\_cell\_directions(FCDs)$ . Έτσι, ο πράκτορας γνωρίζει για μελλοντικές διαδρομές ότι η συγκεκριμένη μετακίνηση είναι ασφαλής.

#### ❖ $blocked\_cell\_directions(BCDs)$

Όπου  $BCDs$  λίστα (αρχικά άδεια). Η πεποίθηση αυτή χρησιμεύει ως εξής: όποτε ο πράκτορας διαπιστώσει ότι η μετακίνηση του ρομπότ από το κελί  $Cell := [Row, Col]$  στο γειτονικό αυτού προς την κατεύθυνση  $Dir$  είναι ανεπιτυχής, τότε προσθέτει το στοιχείο  $[Cell, Dir]$  στην λίστα  $BCDs$  της πεποίθησης  $blocked\_cell\_directions(BCDs)$ . Έτσι, ο πράκτορας γνωρίζει για μελλοντικές διαδρομές ότι από την συγκεκριμένη μετακίνηση θα προκύψει σύγκρουση.

❖ *num\_of\_free\_moves(NoFMs)*

Όπου *NoFMs*, αρχικά 1, είναι το πλήθος των επιτυχών μετακινήσεων που έχει κάνει το ρομπότ. Κάθε φορά που το ρομπότ θα μετακινηθεί από ένα κελί σε γειτονικό αυτού για πρώτη φορά, η τιμή *NoFMs* αυξάνεται κατά μια μονάδα.

❖ *num\_of\_collisions(NoCs)*

Όπου *NoCs*, αρχικά 1, είναι το πλήθος των συγκρούσεων του ρομπότ με τοίχο. Κάθε φορά που το ρομπότ συγκρούεται σε συγκεκριμένο τοίχο για πρώτη φορά, η τιμή *NoCs* αυξάνεται κατά μια μονάδα.

❖ *routes(RTs)*

Όποτε ο πράκτορας λαμβάνει μια αντίληψη *route(R)*, με  $R := [Src, Dst]$ , ο πράκτορας προσθέτει το ζεύγος αφετηρίας-προορισμού του πελάτη ως διαδρομή στη λίστα *RTs* της αντίληψης *routes(RTs)*. Έτσι, ο πράκτορας διατηρεί μια λίστα με τις διαδρομές πελατών. Όποτε ο πράκτορας πιστέψει ότι έχει εξυπηρετήσει τη διαδρομή ενός πελάτη διαγράφει την διαδρομή του από την λίστα.

❖ *plan(PL)*

Όπου *PL* είναι η λίστα που διατηρεί την ακολουθία των διαδοχικών δράσεων του τρέχοντος πλάνου του πράκτορα.

❖ *src(Src)*

Όπου *Src* το χρώμα αφετηρίας της τρέχουσας διαδρομής που εξυπηρετεί ο πράκτορας.

❖ *dst(Dst)*

Όπου *Dst* το χρώμα προορισμού της τρέχουσας διαδρομής που εξυπηρετεί ο πράκτορας.

## Οι Στόχοι του Πράκτορα

Οι βασικοί στόχοι του πράκτορα είναι οι εξής:

✚ *!wait*: ο στόχος του πράκτορα να περιμένει (όσο θεωρεί ότι είναι καλύτερο το να εργαστεί).

✚ *!plan*: ο στόχος του πράκτορα να σχεδιάσει ένα πλάνο δράσης.

✚ *!work*: ο στόχος του πράκτορα να εργαστεί επί του πλάνου δράσης.

✚ *!execute(A)*: ο στόχος του πράκτορα να εκτελέσει μια συγκεκριμένη δράση, *A*.

✚ !*check*: ο στόχος του πράκτορα να εξετάσει το αποτέλεσμα της δράσης μετακίνησης.

Ο αρχικός στόχος του πράκτορα είναι αυτός της αναμονής, !*wait*.

## Τα Σχέδια του Πράκτορα

Ο πράκτορας διαθέτει σχέδια για την πραγματοποίηση οποιουδήποτε στόχου του. Το ποιο σχέδιο επιλέγεται για την πραγματοποίηση του στόχου εξαρτάται από το context του σχεδίου, που είναι ένα σύνολο συνθηκών οι οποίες θα πρέπει να ισχύουν ώστε να επιλεχθεί το αντίστοιχο σχέδιο.

Όταν ο πράκτορας λαμβάνει την αντίληψη περί της διαδρομής νεοεμφανιζόμενου πελάτη *route(R)*, θα πρέπει να προσθέσει την διαδρομή *R* στην λίστα των προς εξυπηρέτηση διαδρομών πελατών, *RTs*, της πεποιθήσης, *routes(RTs)*. Επειδή αυτό το γεγονός σηματοδοτεί στον πράκτορα την αλλαγή της κατάστασης του περιβάλλοντος, στο τέλος του σχεδίου διαχείρισης νέας διαδρομής ο πράκτορας αποκτά τον στόχο να σχεδιάσει το πλάνο του:

```
@route[atomic]
+route(R) : routes(RTs) <-
    .concat(RTs, [R], NewRTs);
    -+routes(NewRTs);
    .print("Routes: ", NewRTs);
    !plan.
```

Ο αρχικός στόχος του πράκτορα είναι αυτός της αναμονής, !*wait*. Τον στόχο αυτόν ο πράκτορας τον διατηρεί όσο πιστεύει ότι είτε δεν έχει επαρκή ενέργεια για να δράσει είτε ότι δεν υπάρχουν διαδρομές πελατών για να εξυπηρετήσει.

```
+!wait : (energy(E) & E <= 0) | (routes(RTs) & .length(RTs, 0)) <-
    wait; !wait.
```

Όπως φαίνεται στο παραπάνω απόσπασμα του κώδικα .asl κατ' αυτό το σχέδιο ο πράκτορας εκτελεί την δράση *wait* και αποκτά ξανά τον στόχο αναμονής !*wait*.

Όταν η συνθήκη αναμονής πάψει να ισχύει, ο πράκτορας αντιμετωπίζει τον στόχο της αναμονής αρχικά με το να αποκτήσει τον στόχο του να σχεδιάσει ένα πλάνο δράσης, !*plan*. Μετά αποκτά τον στόχο του να εργαστεί επί του πλάνου, !*work*. Και τέλος, αφότου ολοκληρώσει το έργο του αποκτά τον στόχο να περιμένει ξανά, !*wait*.

```
+!wait <- !plan; !work; !wait.
```

Όταν ο πράκτορας αποκτά τον στόχο του σχεδιασμού ενός πλάνου δράσης, !*plan*, λαμβάνει υπόψη του, ως το context του σχεδίου, τις εξής πεποιθήσεις του:

0. *free\_cell\_directions(FCDS)*: Γνωστές ελεύθερες ή ασφαλείς μετακινήσεις.
1. *blocked\_cell\_directions(BCDs)*: Γνωστές μπλοκαρισμένες μετακινήσεις ή θέσεις τοίχων.

2. *nodes(NDs)*: Τα κομβικά κελιά, δηλ., το κελί του ρομπότ και τα χρωματιστά κελιά.
3. *routes(RTs)*: Οι ενεργές διαδρομές πελατών.
4. *carry(C)*: Το χρώμα προορισμού του πελάτη που κρατάει το ρομπότ.
5. *energy(E)*: Η τρέχουσα ενέργεια του ρομπότ.
6. *num\_of\_free\_moves(NoFMs)*: Το πλήθος των γνωστών ή ασφαλών μετακινήσεων.
7. *num\_of\_collisions(NoCs)*: Το πλήθος των συγκρούσεων.

Λαμβάνοντας υπόψη τα παραπάνω, το σχέδιο πραγματοποιεί την υλοποιημένη εσωτερική δράση *planner.plan(FCDs,BCDs,NDs,RTs,C,E,NoFMs,NoCs,NewPL)*. Η δράση αυτή επιτελεί τον σχεδιασμό του πλάνου δράσης, βάσει των παραπάνω πεποιθήσεων, και κατόπιν εκτέλεσης της η μεταβλητή *NewPL* θα περιέχει την ακολουθία των δράσεων του πλάνου ως λίστα. Στο τέλος του σχεδίου αυτού, η αντίληψη περί του πλάνου ενημερώνεται με την λίστα του νέου πλάνου *NewPL*.

```
@plan[atomic]
+!plan : free_cell_directions(FCDs) & blocked_cell_directions(BCDs) &
        nodes(NDs) & routes(RTs) & carry(C) & energy(E) &
        num_of_free_moves(NoFMs) & num_of_collisions(NoCs) <-
        planner.plan(FCDs, BCDs, NDs, RTs, C, E, NoFMs, NoCs, NewPL);
        -+plan(NewPL).
```

Όταν ο πράκτορας αποκτά τον στόχο του να εργαστεί επί του πλάνου, *!work*, διακρίνονται τρεις περιπτώσεις σχεδίων για την επιτέλεση του στόχου αυτού:

Στην πρώτη περίπτωση, ισχύουν οι συνθήκες αναμονής, δηλαδή ότι ο πράκτορας πιστεύει πως είτε το ρομπότ δεν έχει ενέργεια είτε δεν υπάρχουν διαδρομές πελατών για να εξυπηρετήσει, όπου τότε ο πράκτορας αποκτά ξανά τον στόχο αναμονής, *!wait*.

```
+!work : (energy(E) & E == 0) | (routes(RTs) & .length(RTs, 0)) <-
        !wait.
```

Στην δεύτερη περίπτωση, εάν το πλάνο είναι άδειο, τότε ο πράκτορας δεν προβαίνει σε καμία λειτουργία, και θεωρείται πως η εργασία έχει ολοκληρωθεί.

```
+!work : plan(PL) & .length(PL, 0).
```

Κατά την τρίτη περίπτωση, το ρομπότ έχει ενέργεια, υπάρχουν διαδρομές πελατών προς εξυπηρέτηση και ο πράκτορας έχει ένα μη-κενό σχέδιο, οπότε προβαίνει στην εκτέλεση του σχεδίου. Η εκτέλεση του σχεδίου γίνεται με την αφαίρεση της κεφαλής του σχεδίου, *A*, ενημέρωση του τρέχοντος σχεδίου ώστε να είναι η ουρά του προηγούμενου, *PLTail*, και την απόκτηση του στόχου της εκτέλεσης της δράσης *A*, *!execute(A)*. Κατόπιν αυτών, ο πράκτορας διατηρεί τον στόχο του να εργαστεί επί του πλάνου, *!work*.

```
+!work : plan([A | PLTail]) <-
        -+plan(PLTail);
        !execute(A);
        !work.
```

Ακολούθως παρατίθεται ο πίνακας αριθμητικής αναπαράστασης των δράσεων, η οποία αναπαράσταση χρησιμοποιείται από τον σχεδιαστή δράσης, *planner*, και με την οποία το πλάνο αποθηκεύεται στην λίστα *PL* της πεποίησης *plan(PL)*.

| Αναπαράσταση δράσης, <i>A</i> | Δράση           | Περιγραφή                                      |
|-------------------------------|-----------------|------------------------------------------------|
| 0                             | <i>move</i> (0) | Μετακίνηση προς την κατεύθυνση 0 (δεξιά).      |
| 1                             | <i>move</i> (1) | Μετακίνηση προς την κατεύθυνση 1 (πάνω).       |
| 2                             | <i>move</i> (2) | Μετακίνηση προς την κατεύθυνση 2 (αριστερά).   |
| 3                             | <i>move</i> (3) | Μετακίνηση προς την κατεύθυνση 3 (κάτω).       |
| 4                             | <i>unload</i>   | Εκφόρτωση πελάτη.                              |
| 5                             | <i>load</i> (1) | Φόρτωση πελάτη με χρώμα προορισμού 1 (κόκκινο) |
| 6                             | <i>load</i> (2) | Φόρτωση πελάτη με χρώμα προορισμού 2 (πράσινο) |
| 7                             | <i>load</i> (3) | Φόρτωση πελάτη με χρώμα προορισμού 3 (μπλε)    |
| 8                             | <i>load</i> (4) | Φόρτωση πελάτη με χρώμα προορισμού 4 (κίτρινο) |
| 9                             | <i>wait</i>     | —                                              |

Ο παραπάνω πίνακας μπορεί να συνοψιστεί όπως φαίνεται παρακάτω:

| Αριθμητική αναπαράσταση δράσης, <i>A</i> | Δράση                       | Περιγραφή                      |
|------------------------------------------|-----------------------------|--------------------------------|
| <i>A</i> = 0, 1, 2, 3                    | <i>move</i> ( <i>A</i> )    | Μετακίνηση προς <i>A</i> .     |
| <i>A</i> = 4                             | <i>unload</i>               | Εκφόρτωση πελάτη.              |
| <i>A</i> = 5, 6, 7, 8                    | <i>load</i> ( <i>A</i> - 4) | Φόρτωση πελάτη ( <i>A</i> - 4) |
| 9                                        | <i>wait</i>                 | —                              |

Όταν ο πράκτορας αποκτά τον στόχο της εκτέλεσης της δράσης *A*, *!execute(A)*, θα εκτελέσει διαφορετικό σχέδιο κυρίως βάσει της τιμής της δράσης *A*. Στην πρώτη περίπτωση πρόκειται για την δράση μετακίνησης, όπου  $0 \leq A \leq 3$ . Κατ' αυτήν, ο πράκτορας λαμβάνοντας υπόψη του το τρέχον κελί *Cell*, στο οποίο βρίσκεται το ρομπότ μέσω της αντίληψης *robot(Cell)*, καταγράφει το κελί αυτό ως την θέση του ρομπότ πριν εκτελέσει την δράση της μετακίνησης,  $- + robot(Cell, premove)$ . Επίσης καταγράφει την κατεύθυνση μετακίνησης, ως  $- + direction(Dir)$ . Ύστερα, εκτελεί την δράση μετακίνησης, *move*, κατόπιν της εκτέλεσης της οποίας αποκτά τον στόχο του να εξετάσει το αποτέλεσμα της δράσης μετακίνησης, *!check*.

```
@move
+!execute(A) : 0 <= A & A <= 3 & robot(Cell) <-
```

```

-+robot(Cell, premove);
-+direction(A);
move(A);
!check.

```

Στην δεύτερη περίπτωση, πρόκειται για την εκτέλεση δράσης εκφόρτωσης πελάτη, όπου  $A = 4$ . Η δράση αυτή όταν πραγματοποιείται ολοκληρώνει την εξυπηρέτηση διαδρομής πελάτη, επομένως, θα πρέπει να διαγραφεί η εν λόγω διαδρομή από την λίστα *RTs* της πεποιθήσης *routes(RTs)*. Για τον λόγο αυτό, κατά το σχέδιο αυτό λαμβάνεται υπόψη, στο context, η πεποιθήση *routes(RTs)*, καθώς και οι πεποιθήσεις της τρέχουσας αφετηρίας και προορισμού, *src(Src)* και *dst(Dst)*. Ο πράκτορας εκτελεί την δράση εκφόρτωσης, *unload*, και αφαιρεί τη διαδρομή. Τέλος αποκτάει τον στόχο να σχεδιάσει ξανά το πλάνο δράσης του. (Σημείωση: η αφαίρεση της ολοκληρωμένης διαδρομής από την λίστα των διαδρομών είναι υλοποιημένη και αυτή ως στόχος με το αντίστοιχο σχέδιο, αλλά δεν θα αναλυθεί στο παρόν έγγραφο.)

```

@unload[atomic]
+!execute(A) : A == 4 & routes(RTs) & src(Src) & dst(Dst) <-
  unload;
  !remove_route([Src, Dst], RTs);
  !plan.

```

Κατά την τρίτη περίπτωση πρόκειται για την δράση της φόρτωσης πελάτη, με  $5 \leq A \leq 8$ . Ο πράκτορας εκτελεί την δράση φόρτωσης του πελάτη ( $A - 4$ ), όπως φαίνεται στον προηγούμενο πίνακα της αναπαράστασης των δράσεων. Στη συνέχεια θέτει τη τρέχουσα αφετηρία και προορισμό της διαδρομής που θα εξυπηρετήσει με την φόρτωση του εν λόγω πελάτη. (Σημείωση: η ενημέρωση της αφετηρίας τρέχουσας διαδρομής έχει υλοποιηθεί ως στόχος με το αντίστοιχο σχέδιο, αλλά δεν θα αναλυθεί στο παρόν έγγραφο.)

```

@load[atomic]
+!execute(A) : 5 <= A & A <= 8 <-
  load(A - 4);
  !src;
  -+dst(A - 4).

```

Μπορεί ο σχεδιαστής δράσης να αποφανθεί ότι υπό την τρέχουσα όψη επί της κατάστασης του περιβάλλοντος του πράκτορα, δεν συμφέρει να προβεί σε καμία δράση. Τότε το πλάνο αποτελείται από μόνο ένα στοιχείο που αντιστοιχεί στην δράση αναμονής, *wait*. Ο σχεδιαστής του πλάνου βασίζει τον σχεδιασμό του πάνω σε μια πιθανότητα σύγκρουσης, η οποία είναι απλά ο λόγος  $NoFMs \div (NoFMs + NoCs)$ . Η πιθανότητα αυτή καθορίζει το ρίσκο που επιφέρει μια νέα μετακίνηση. Εάν ο σχεδιαστής θεωρεί το ρίσκο αρκετά μεγάλο, μπορεί να επιστρέφει συνεχώς την τιμή 9. Τότε όμως ο πράκτορας ενδεχομένως να μη προβεί ποτέ σε καμία δράση. Για τον λόγο αυτό, κάθε φορά που ο πράκτορας εκτελεί την δράση αναμονής όταν αυτή προέρχεται από το πλάνο, προσauξάνει την τιμή *NoFMs* κατά μια μονάδα, ώστε το ρίσκο να μειώνεται οργανικά, με κάθε τέτοια αναμονή, επαρκώς ώστε να προβεί ο πράκτορας σε δράση, αλλά όχι παραπάνω από αυτό.

```

@wait

```

```

+!execute(A) : num_of_free_moves(NoFMs) <-
  .print("Searching for a viable plan...");
  .print("Decreasing new-move risk...");
  +num_of_free_moves(NoFMs + 1);
wait.

```

Κατόπιν της εκτέλεσης της δράσης μετακίνησης, *move*, ο πράκτορας αποκτά τον στόχο του να εξετάσει το αποτέλεσμα της δράσης μετακίνησης, *!check*. Κατ' αυτήν, εάν ο πράκτορας πιστεύει ότι το ρομπότ παρέμεινε στο ίδιο κελί, τότε συνάγει ότι προέκυψε σύγκρουση του ρομπότ με τοίχο. Σε αυτήν την περίπτωση, εάν πρόκειται για νέα σύγκρουση, δηλαδή είναι η πρώτη φορά που ο πράκτορας συγκρούεται με τον τοίχο αυτόν, τότε η κατεύθυνση προς την οποία κινήθηκε από το τρέχον κελί το ρομπότ προστίθεται ως στοιχείο στην λίστα των μπλοκαρισμένων κατευθύνσεων κελιών *BCDs*. Επιπλέον, προσαυξάνεται κατά μια μονάδα ο μετρητής συγκρούσεων, και τέλος ο πράκτορας αποκτά τον στόχο του να επανασχεδιάσει το πλάνο του, αφού το τρέχον σχέδιο δεν είναι ορθό λόγω της δράσης μετακίνησης, η οποία απέτυχε λόγω της σύγκρουσης.

```

@check_collision[atomic]
+!check : robot(Cell, premove) & robot(Cell) & direction(Dir) &
  blocked_cell_directions(BCDs) &
  not .member([Cell, Dir], BCDs) &
  num_of_collisions(NoCs) <-
  .concat(BCDs, [[Cell, Dir]], NewBCDs);
  +blocked_cell_directions(NewBCDs);
  +num_of_collisions(NoCs + 1);
!plan.

```

Στην δεύτερη περίπτωση, εάν το ρομπότ δεν βρίσκεται στο ίδιο κελί με πριν τη μετακίνηση, και η μετακίνηση αυτή είναι νέα, τότε αυτή προστίθεται στην λίστα *FCDs* και ο μετρητής ελεύθερων κατευθύνσεων κελιών προσαυξάνεται.

```

@check_free[atomic]
+!check : robot(Cell, premove) & direction(Dir) &
  free_cell_directions(FCDs) &
  not .member([Cell, Dir], FCDs) &
  num_of_free_moves(NoFMs) <-
  .concat(FCDs, [[Cell, Dir]], NewFCDs);
  +free_cell_directions(NewFCDs);
  +num_of_free_moves(NoFMs + 1).
+!check.

```

Τέλος, εάν δεν πρόκειται για νέα μετακίνηση ή σύγκρουση, τότε ο πράκτορας δεν κάνει κάτι κατά την εξέταση της μετακίνησης:

```

+!check.

```

## Ο Σχεδιαστής Πλάνου του Πράκτορα

Ο σχεδιασμός του πλάνου δράσης του πράκτορα διακρίνεται σε δύο προβλήματα. Το ένα είναι αυτό της εύρεσης της ακολουθίας μετακινήσεων για την μετάβαση μεταξύ δύο αυθαίρετων κελιών του πλέγματος, στο οποίο θα αναφερόμαστε ως πρόβλημα καθοδήγησης, *navigation*. Το δεύτερο είναι αυτό του καθορισμού της σειράς εξυπηρέτησης των διαδρομών των πελατών, στο οποίο θα αναφερόμαστε ως το πρόβλημα της δρομολόγησης, *routing*. Ο αλγόριθμος δρομολόγησης χρησιμοποιεί τον αλγόριθμο καθοδήγησης και συνθέτει το τελικό πλάνο.

### Το Πρόβλημα και ο Αλγόριθμος της Καθοδήγησης (*Navigation*)

Για το πρόβλημα της καθοδήγησης, η κατάσταση είναι οι συντεταγμένες πλέγματος του κελιού στο οποίο θα βρίσκεται το ρομπότ (*row*, *col*),

Στο πρόβλημα της καθοδήγησης, υπό τη δοθείσα περιγραφή, ορίζεται το κόστος μετάβασης ως το κόστος της δράσης της μετακίνησης,  $ACTION\_COST := 1$ . Επιπλέον, ορίζεται το κόστος σύγκρουσης ως  $COLLISION\_COST := 100$ .

Το πρόβλημα καθοδήγησης περιλαμβάνει επιπλέον τις εξής παραμέτρους:

- Έναν Boolean πίνακα *FCDs* ο οποίος έχει την τιμή *true* στην θέση *FCDs[row][col][dir]*, εάν είναι γνωστό πως είναι ασφαλής η μετακίνηση από το κελί με συντεταγμένες πλέγματος (*row*, *col*) προς την κατεύθυνση *dir*.
- Έναν Boolean πίνακα *BCDs* ο οποίος έχει την τιμή *true* στην θέση *BCDs[row][col][dir]*, εάν είναι γνωστό ότι κατά τη μετακίνηση από το κελί με συντεταγμένες πλέγματος (*row*, *col*) προς την κατεύθυνση *dir* θα προκύψει σύγκρουση.
- Την πιθανότητα σύγκρουσης, *collisionProb*.
- Την αρχική κατάσταση, δηλαδή το κελί στο οποίο βρίσκεται αρχικά το ρομπότ, *initState*.
- Την κατάσταση στόχο, δηλαδή το κελί στο οποίο είναι επιθυμητό να βρεθεί το ρομπότ, *goalState*.

Ως δράση ή μετάβαση θεωρείται η δράση της μετακίνησης, η οποία καθορίζεται από μια κατεύθυνση. Έστω η κατάσταση  $state := (row, col)$ , και η δράση μετακίνησης προς την κατεύθυνση *dir*. Η κατάσταση που προκύπτει με τη μετάβαση *dir* από την κατάσταση *state* θα είναι:

$$result \leftarrow \begin{cases} null & , \text{ αν } BCDs[row][col][dir] \\ state + DS[d], & \text{ αλλιώς} \end{cases}$$

Με άλλα λόγια, εάν η κατεύθυνση *dir* του κελιού στο οποίο βρίσκεται το ρομπότ είναι μπλοκαρισμένη από τοίχο, τότε το αποτέλεσμα της μετακίνησης θεωρείται κενό, δηλαδή η

κατάσταση δεν έχει γείτονα μέσω αυτής της μετάβασης. Αλλιώς εάν δεν είναι γνωστό πως η κατεύθυνση είναι μπλοκαρισμένη, τότε επιστρέφεται το γειτονικό κελί προς την κατεύθυνση αυτή.

Το κόστος μετάβασης εξαρτάται από τους πίνακες,  $FCDs$ ,  $BCDs$ , καθώς και από την πιθανότητα σύγκρουσης. Συγκεκριμένα, εάν για την κατάσταση  $state := (row, col)$  εξετάζεται η δράση μετάβασης μετακίνησης προς την κατεύθυνση  $dir$ , και εάν είναι γνωστό πως αυτή είναι ασφαλής, τότε το κόστος μετάβασης είναι  $ACTION\_COST$ , αλλιώς εάν είναι γνωστό πως αποτελεί σύγκρουση, τότε το κόστος μετάβασης είναι  $ACTION\_COST + COLLISION\_COST$ . Αλλιώς είναι άγνωστο το εάν η μετάβαση είναι ασφαλής ή όχι, οπότε το κόστος μετάβασης θα είναι  $ACTION\_COST + risk$ , όπου το ρίσκο είναι  $risk = collisionProb \times COLLISION\_COST$ . Δηλαδή:

$$stepCost \leftarrow \begin{cases} ACTION\_COST, & \text{αν } FCDs[row][col][dir] \\ ACTION\_COST + COLLISION\_COST, & \text{αν } FCDs[row][col][dir] \\ ACTION\_COST + collisionProb \times COLLISION\_COST, & \text{αλλιώς} \end{cases}$$

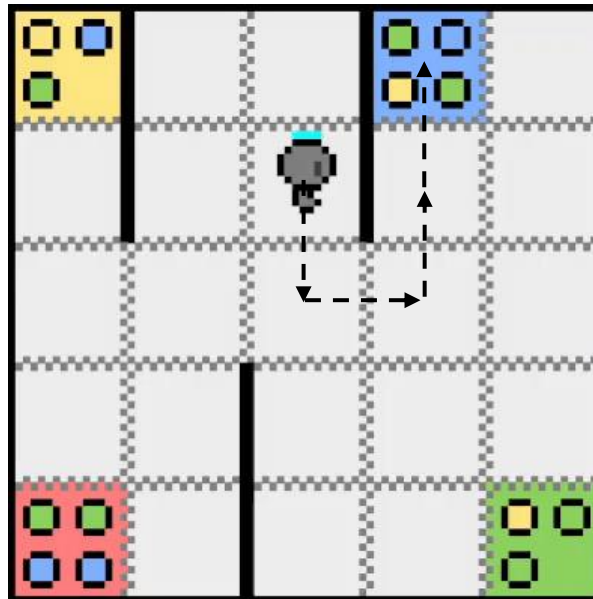
Επιπλέον, για το πρόβλημα της καθοδήγησης γίνεται η χρήση της ευριστικής συνάρτησης Manhattan, ως η απόσταση της υπό εξέταση κατάστασης  $state := (row, col)$  με την κατάσταση στόχου,  $goalState := (goalRow, goalCol)$ .

$$h(state) := |goalRow - row| + |goalCol - col|$$

Ως κόμβος αναζήτησης χρησιμοποιείται ο κλασσικός ορισμός του κόμβου αναζήτησης γραφήματος. Δηλαδή, ένας κόμβος αναζήτησης περιλαμβάνει τα εξής χαρακτηριστικά:

- Μια κατάσταση,  $n.state$ .
- Έναν γονικό κόμβο,  $n.parent$ .
- Την δράση ή μετάβαση μέσω της οποίας προέκυψε αυτός ο κόμβος από τον γονέα του,  $n.action$ .
- Το κόστος της διαδρομής από τη ρίζα του δέντρου αναζήτησης μέχρι τον τρέχον κόμβο,  $n.pathCost$ , ή και  $g(n)$ .
- Και για  $A^*$  αναζήτηση, συμπεριλαμβάνεται η τιμή εκτιμώμενου κόστους ως το άθροισμα του κόστους μονοπατιού με την ευριστική τιμή για την κατάσταση που ενθυλακώνει ο κόμβος, δηλαδή,  $n.evalCost := g(n) + h(n.state)$ .

Ο αλγόριθμος της καθοδήγησης, όπως ορίστηκε προηγουμένως, επιλύεται με τον αλγόριθμο  $A^*$ . Η λύση αποτελείται από μια ακολουθία κατευθύνσεων μετακινήσεων, με την αριθμητική τους αναπαράσταση, δηλαδή, 0 για δεξιά, 1 για επάνω, 2 για αριστερά και 3 για κάτω. Για παράδειγμα, παρατίθεται η παρακάτω κατάσταση μιας εκτέλεσης κατά την οποία ο πράκτορας θα κατευθύνει το ρομπότ από το αρχικό κελί, (1,2), στο μπλε κελί, βάσει της ακολουθίας κατευθύνσεων μετακινήσεων που επιστρέφει ο αλγόριθμος καθοδήγησης: («κάτω», «δεξιά», «πάνω», «πάνω»)

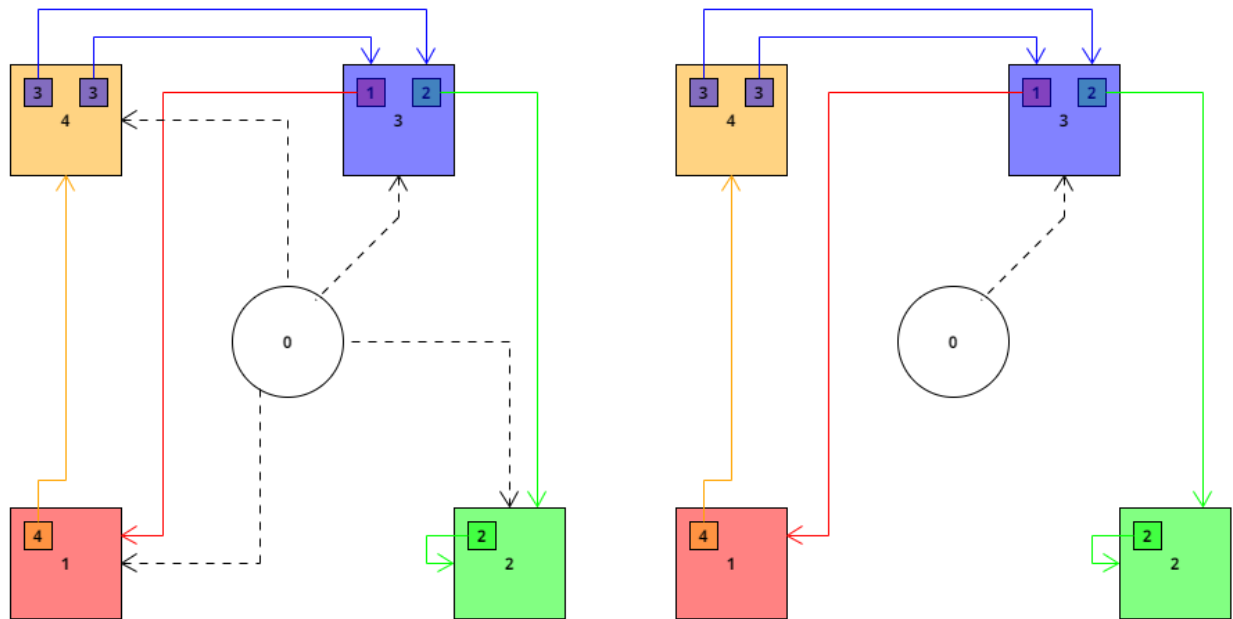
$$NavSolution := (12, 3, 0, 1, 1)$$


, όπου η πρώτη τιμή της λύσης δεν αναπαριστά δράση, αλλά το αναμενόμενο κόστος της λύσης.

## Το Πρόβλημα και ο Αλγόριθμος της Δρομολόγησης (Router)

Το πρόβλημα της δρομολόγησης αφορά στην εύρεση της σειράς ή ακολουθίας των εξυπηρετήσεων η οποία θα ελαχιστοποιεί το αναμενόμενο κόστος. Πρόκειται για πρόβλημα αναζήτησης το οποίο εδώ μοντελοποιείται με ένα γράφημα. Τους κόμβους του γραφήματος αποτελούν τα χρωματιστά κελιά καθώς και το τρέχον κελί στο οποίο βρίσκεται το ρομπότ. Οι κόμβοι αναπαρίστανται με το χρώμα τους, 0 για το κελί του ρομπότ, 1 για το κόκκινο κελί, 2 για το πράσινο κελί, 3 για το μπλε κελί και 4 για το κίτρινο κελί.

Η παρουσία ενός πελάτη με χρώμα προορισμού *dst* σε ένα κελί-κόμβο χρώματος *src* αντιστοιχεί σε μια κατευθυνόμενη ακμή μεταξύ του κελιού-κόμβου *src* και του κελιού-κόμβου *dst*. Εάν το ρομπότ δε κρατά πελάτη τότε θεωρείται πως υπάρχει μια ακμή από το κελιού-κόμβου του ρομπότ προς κάθε άλλο κελί-κόμβο. Εάν το ρομπότ κρατά πελάτη με χρώμα προορισμού *dst*, τότε θεωρείται πως υπάρχει μόνο μια ακμή από το ρομπότ προς το κελί-κόμβο χρώματος *dst*. Δίνονται δύο αναπαραστάσεις του γραφήματος για την κάθε περίπτωση:



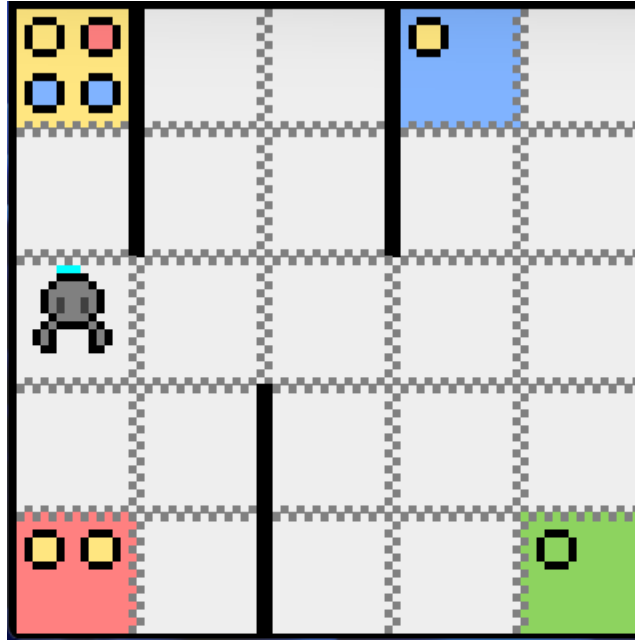
Στο παραπάνω σχήμα, αριστερά είναι το γράφημα του προβλήματος δρομολόγησης για μια συγκεκριμένη κατάσταση όπου το ρομπότ δεν κρατάει πελάτη, ενώ το δεξί γράφημα αντιστοιχεί στην ίδια κατάσταση με τη διαφορά ότι το ρομπότ κρατά πελάτη με προορισμό το μπλε κελί-κόμβο.

Ο αλγόριθμος επίλυσης του προβλήματος δρομολόγησης καλείται για να βρει τη βέλτιστη ακολουθία μεταβάσεων μεταξύ των κόμβων του παραπάνω γραφήματος. Κάθε φορά που γίνεται μετάβαση, εξυπηρετείται μια διαδρομή πελάτη και επομένως η μετάβαση αυτή δεν θα είναι διαθέσιμη σε μεταγενέστερες καταστάσεις των κόμβων της αναζήτησης. Το κόστος της μετάβασης από το ένα κελί-κόμβο στο άλλο καθορίζεται από τον αλγόριθμο καθοδήγησης μεταξύ εν λόγω κελιών-κόμβων.

Η κατάσταση, *state*, για ένα πρόβλημα δρομολόγησης ορίζει τις εξής παραμέτρους:

- Το χρώμα του κελιού-κόμβου στο οποίο θα βρίσκεται το ρομπότ, *state.color*.
- Το βάθος του δέντρου αναζήτησης, ή και τον αύξοντα αριθμό μετάβασης, *state.depth*
- Την ενέργεια που θα έχει το ρομπότ όταν φθάνει σε αυτήν την κατάσταση, *state.energy*.
- Τον πίνακα γειτνίασης που αναπαριστά το γράφημα των μεταβάσεων, μέσω των ενεργών προς εξυπηρέτηση διαδρομών πελατών, του προβλήματος στην τρέχουσα κατάσταση, *state.routes*.

Συγκεκριμένα, για τον πίνακα γειτνίασης, *routes*, παρατίθεται το ακόλουθο παράδειγμα μιας τρέχουσας κατάστασης εκτέλεσης και του αντίστοιχου πίνακα, στον οποίο απαριθμείται το πλήθος των πελάτων με αφετηρία το κελί-κόμβο χρώματος *src* και προορισμό το κελί-κόμβο χρώματος *dst*.



| Σε, <i>dst</i> \ Από, <i>src</i> | Ρομπότ, 0 | Κόκκινο, 1 | Πράσινο, 2 | Μπλε, 3 | Κίτρινο, 4 |
|----------------------------------|-----------|------------|------------|---------|------------|
| Ρομπότ, 0                        | 0         | 1          | 1          | 1       | 1          |
| Κόκκινο, 1                       | 0         | 0          | 0          | 0       | 2          |
| Πράσινο, 2                       | 0         | 0          | 1          | 0       | 0          |
| Μπλε, 3                          | 0         | 0          | 0          | 0       | 1          |
| Κίτρινο, 4                       | 0         | 1          | 0          | 2       | 1          |

Στο πρόβλημα της δρομολόγησης, υπό τη δοθείσα περιγραφή, ορίζεται η αμοιβή εξυπηρέτησης ίση με  $REWARD := 20$ .

Το πρόβλημα καθοδήγησης περιλαμβάνει επιπλέον τις εξής παραμέτρους:

- Έναν πίνακα *nodes* ο οποίος αντιστοιχίζει το χρώμα του κάθε κελιού-κόμβου στις συντεταγμένες του στο πλέγμα.
- Την αναπαράσταση του πελάτη που κρατά το ρομπότ, *carry*, 0 αν δεν κρατά πελάτη.
- Την πιθανότητα σύγκρουσης, *collisionProb*, η οποία υπολογίζεται ως το εξής πηλίκο  $NoFMs \div (NoFMs + NoCs)$ .
- Την αρχική κατάσταση με την οποία έχει κληθεί ο αλγόριθμος, *initState*.

Κατά την αρχικοποίηση του προβλήματος, ο αλγόριθμος δρομολόγησης επικαλείται τον αλγόριθμο καθοδήγησης για κάθε ζεύγος κελιών-κόμβων του προβλήματος που συνδέονται με τουλάχιστον μια ακμή. Έτσι, προκύπτουν τα κόστη μεταβάσεων για το πρόβλημα, αλλά και οι ενδιάμεσες ακολουθίες

μετακινήσεων *navs*. Ουσιαστικά, κάθε μετάβαση στο πρόβλημα δρομολόγησης ενθυλακώνει μια λύση του αλγορίθμου καθοδήγησης, *nav*, για την μετάβαση από το ένα κελί-κόμβο στο άλλο.

Έστω μια κατάσταση του προβλήματος δρομολόγησης, *state*, και μια μετάβαση, η οποία ορίζεται από το χρώμα του κελιού-κόμβου στο οποίο θα γίνει η μετάβαση, *t*. Η κατάσταση που προκύπτει από την μετάβαση αλλάζει ως εξής:

- Εάν πρόκειται για την πρώτη μετάβαση και το ρομπότ δεν κρατάει πελάτη, τότε η ενέργεια του ρομπότ στην επόμενη κατάσταση θα είναι αυτή της προηγούμενης μείον το πλήθος των μετακινήσεων για την εν λόγω μετάβαση.
- Εάν πρόκειται για την πρώτη μετάβαση και το ρομπότ κρατάει πελάτη, τότε η ενέργεια του ρομπότ στην επόμενη κατάσταση είναι αυτή της προηγούμενης μειωμένη κατά το πλήθος των μετακινήσεων αλλά και μια ακόμη μονάδα ενέργειας για την δράση της εκφόρτωσης του πελάτη που κρατούσε το ρομπότ.
- Εάν δεν πρόκειται για την πρώτη μετάβαση, τότε το ρομπότ θα πρέπει να φορτωθεί με τον πελάτη του προορισμού (μια μονάδα ενέργειας) να μεταβεί στο κελί-κόμβο προορισμού του πελάτη και να εκφορτώσει τον πελάτη. Άρα, η ενέργεια της επόμενης κατάστασης για το ρομπότ θα είναι αυτή της προηγούμενης μειωμένη κατά το πλήθος των μετακινήσεων μέχρι τον προορισμό και κατά τις δύο μονάδες ενέργειας για την φόρτωση και εκφόρτωση του πελάτη.

Εάν η ενέργεια που προκύπτει για την επόμενη κατάσταση είναι αρνητική τότε η επόμενη κατάσταση είναι κενή, δηλαδή η μετάβαση αυτή θεωρείται αδύνατη. Επιπλέον, αφού η εν λόγω μετάβαση χρησιμοποιήθηκε, η διαδρομή του αντίστοιχου πελάτη εξυπηρετήθηκε, επομένως η αντίστοιχη τιμή του πίνακα γειτνίασης μειώνεται κατά μια μονάδα, για την νέα κατάσταση. Και βέβαια, το χρώμα του κελιού-κόμβου στο οποίο βρίσκεται πλέον το ρομπότ θα είναι *t*.

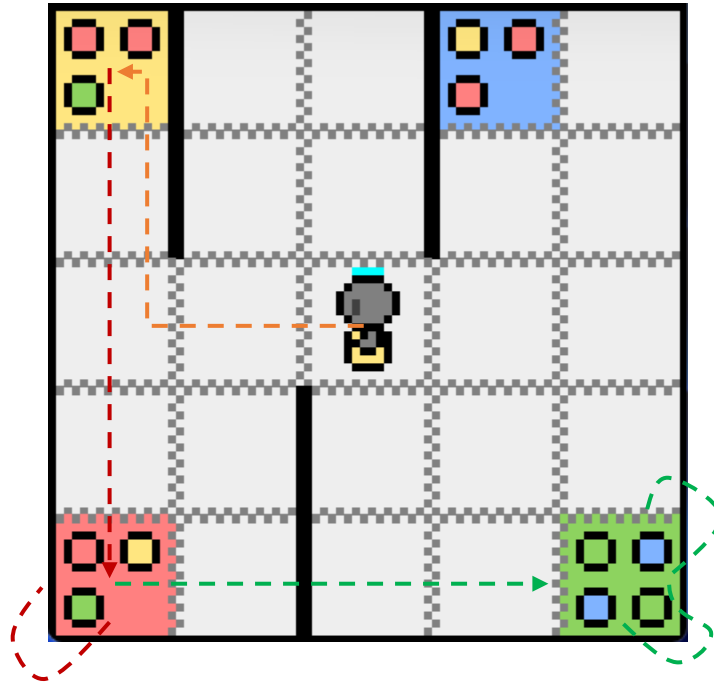
Το κόστος μετάβασης είναι το κόστος, *navCost*, που συνοδεύει την λύση καθοδήγησης μεταξύ του κελιού-κόμβου χρώματος *state.color* και του κελιού-κόμβου χρώματος *t*, για την εν λόγω μετάβαση. Επιπρόσθετα:

- Εάν πρόκειται για την πρώτη μετάβαση και το ρομπότ κρατά πελάτη τότε στο κόστος προστίθεται μια μονάδα για την δράση της εκφόρτωσης του πελάτη, αλλά επειδή κατά την εκφόρτωση θεωρείται ότι ολοκληρώνεται η εξυπηρέτηση του πελάτη, το κόστος θα μειωθεί κατά το *REWARD*.
- Αλλιώς, εάν δεν πρόκειται για την πρώτη μετάβαση, τότε το κόστος προσαυξάνεται κατά δύο μονάδες για τις δράσεις φόρτωσης και εκφόρτωσης πελάτη, και μειώνεται κατά το *REWARD*.

Δηλαδή, πιο συνοπτικά, το κόστος μετάβασης υπολογίζεται ως εξής:

$$stepCost(state, t) \leftarrow \begin{cases} navCost, & \text{αν } state.depth = 0 \text{ και } carry = 0 \\ navCost + 1 - REWARD, & \text{αν } state.depth = 0 \text{ και } carry > 0 \\ navCost + 2 - REWARD, & \text{αλλιώς} \end{cases}$$

Ο αλγόριθμος της δρομολόγησης, όπως έχει οριστεί παραπάνω, επιστρέφει ως λύση την ακολουθία των χρωμάτων των κελιών-κόμβων, με την σειρά που θα πρέπει να τα επισκεφτεί το ρομπότ. Για παράδειγμα, παρατηρήστε την κατάσταση της ακόλουθης εικόνας:



Το δρομολόγιο που επιστρέφει ο αλγόριθμος δρομολόγησης στην περίπτωση του στιγμιότυπου της παραπάνω εκτέλεσης είναι  $RoutingSolution := (4, 1, 1, 2, 2, 2)$ , που ερμηνεύεται ως εξής:

- Μετάβαση στο κίτρινο κελί (4), για εκφόρτωση του κίτρινου πελάτη που κρατά το ρομπότ.
- Φόρτωση του κόκκινου πελάτη και μετάβαση στο κόκκινο κελί (1) για εκφόρτωση του κόκκινου πελάτη.
- Φόρτωση του κόκκινου πελάτη που βρίσκεται στο κόκκινο κελί (κενή μετάβαση στο κόκκινο κελί (1)) και εκφόρτωση του πελάτη σε αυτό.
- Φόρτωση του πράσινου πελάτη που βρίσκεται στο κόκκινο κελί, μετάβαση στο πράσινο κελί (2) και εκφόρτωση του πελάτη.
- Φόρτωση του πράσινου πελάτη που βρίσκεται στο πράσινο κελί (κενή μετάβαση στο πράσινο κελί (2)) και εκφόρτωση του πελάτη σε αυτό.
- Φόρτωση του πράσινου πελάτη που βρίσκεται στο πράσινο κελί (κενή μετάβαση στο πράσινο κελί (2)) και εκφόρτωση του πελάτη σε αυτό.

Ο αλγόριθμος θα επιστρέψει το δρομολόγιο με το ελάχιστο αναμενόμενο κόστος. Εάν δε βρεθεί καμία λύση που να έχει αρνητικό κόστος, δηλαδή κέρδος, τότε ο αλγόριθμος θα επιστρέψει την τιμή

9, υποδεικνύοντας στον πράκτορα ότι είναι καλύτερο να μειώσει το τρέχον ρίσκο νέας μετακίνησης και να μην δράσει σε αυτόν τον κύκλο, με αποτέλεσμα ο πράκτορας να προβεί σε δράση αναμονής.

Κατόπιν της εύρεσης του δρομολογίου, ο αλγόριθμος θα κατασκευάσει το πλήρες πλάνο του πράκτορα, *plan*. Σε πρώτη φάση εισάγει στο πλάνο την ακολουθία μετακινήσεων από το αρχικό κελί του ρομπότ μέχρι το πρώτο κελί-κόμβο της λύσης. Στη συνέχεια εάν το ρομπότ κατά τη πρώτη μετάβαση αυτή κρατούσε πελάτη, εισάγει την τιμή 4 που αναπαριστά εκφόρτωση πελάτη.

Στη συνέχεια για κάθε επόμενο προορισμό, *dst*, εισάγει στο πλάνο πρώτα την τιμή ( $4 + dst$ ) που αναπαριστά την φόρτωση πελάτη με προορισμό *dst*. Μετά εισάγει την ακολουθία μετακινήσεων καθοδήγησης από το τρέχον κελί-κόμβο μέχρι το κελί *dst* και προσθέτει την τιμή 4 για εκφόρτωση.

Για παράδειγμα, το πλάνο για την κατάσταση του παραπάνω παραδείγματος, υπό τις τρέχουσες πεποιθήσεις του πράκτορα, θα είναι το εξής:

$$Plan := (3, 2, 2, 2, 1, 1, 4, 5, 3, 3, 3, 3, 4, 5, 4, 6, 0, 0, 0, 0, 4, 6, 4, 6, 4)$$

## Πειραματικά Αποτελέσματα & Συμπεράσματα

Η Αξιολόγηση του Πράκτορα, *evaluation*, υπολογίζεται ως εξής:

$$evaluation := numServed \times REWARD + numActions \times ACTION + numCollisions \times COLLISION$$

, όπου:

- $COLLISION := -100$ : Το κόστος σύγκρουσης
- $ACTION := -1$ : Το κόστος δράσης
- $REWARD := +20$ : Η αμοιβή εξυπηρέτησης

(Η συμμετοχή του κόστους του πέναλτι λόγω εσφαλμένης εξυπηρέτησης στον υπολογισμό του *evaluation* έχει παραληφθεί, επειδή ο πράκτορας ποτέ δεν θα κάνει εσφαλμένη εξυπηρέτηση, δηλαδή ποτέ δεν αφήνει έναν πελάτη που κρατά, παρά μόνο στον προορισμό του.)

Υπό τα δοθέντα κόστη και αμοιβές, με αντικατάσταση, η έκφραση για την αξιολόγηση είναι:

$$evaluation := 20 \times numServed - numActions - 100 \times numCollisions$$

Κατά μέσο όρο, το πλήθος των δράσεων που απαιτούνται ανά εξυπηρέτηση με πλήρη γνώση του περιβάλλοντος, από έναν βέλτιστο πράκτορα, δεδομένου ότι οι πελάτες εμφανίζονται με τυχαίο και ομοιόμορφο τρόπο, είναι περίπου 6. Οπότε, εάν θέλουμε να πάρουμε μια απλοποιημένη προσέγγιση για την παραπάνω στην περίπτωση ενός βέλτιστου πράκτορα, μπορούμε να γράψουμε:

$$\begin{aligned} evaluation_{opt} &:= (20 - 6) \times numServed - 100 \times numCollisions \\ &:= 14 \times numServed - 100 \times numCollisions \end{aligned}$$

Δηλαδή, για έναν βέλτιστο πράκτορα, μια σύγκρουση ισοδυναμεί (ως προς την αξιολόγηση του πράκτορα) με  $100 \div 14 \approx 7$  εξυπηρετήσεις πελατών.

Προφανώς, για έναν μη-βέλτιστο πράκτορα ο οποίος μάλιστα δεν έχει αρχική γνώση για τις θέσεις των τοίχων, η σύγκρουση έχει αρκετά μεγαλύτερο βάρος ως προς την αξιολόγησή του.

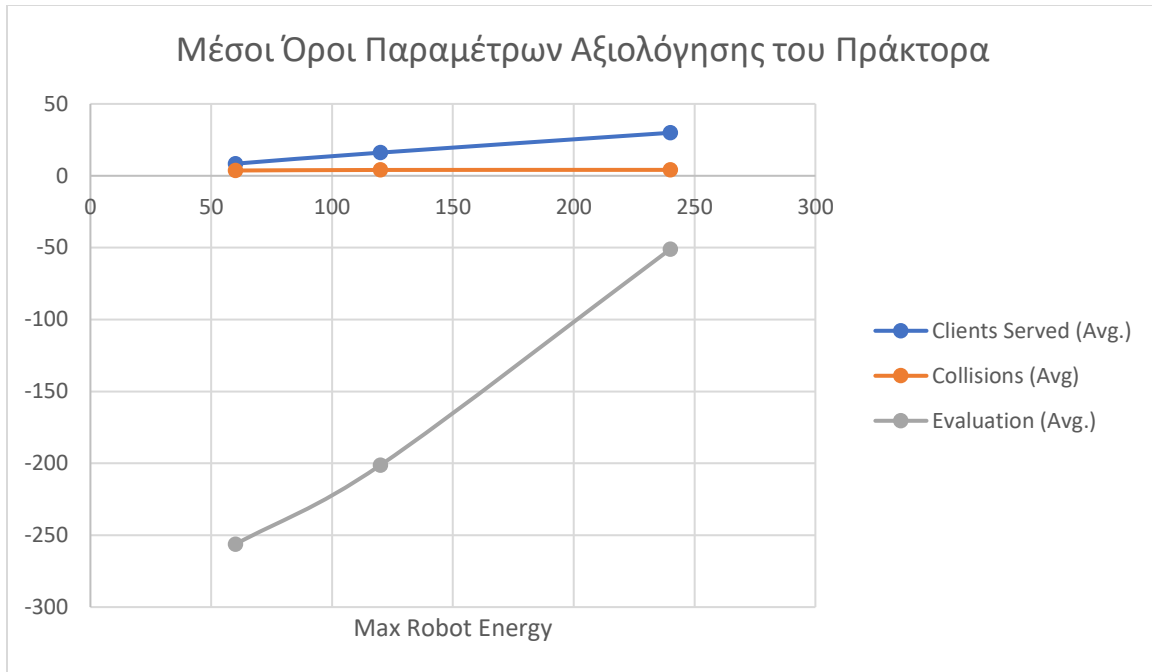
Για τον περιορισμό μελλοντικών συγκρούσεων, το μοντέλο πράκτορα που έχει υλοποιηθεί παρουσιάζει «προτίμηση» ως προς τις διαδρομές που αποτελούνται από γνωστές ασφαλείς μετακινήσεις. Για τον παρατηρητή της προσομοίωσης, θα είναι εμφανές σε ορισμένες περιπτώσεις ότι ο πράκτορας δεν επιλέγει τη βέλτιστη διαδρομή. Αλλά, ξανά, ο πράκτορας δεν γνωρίζει τις θέσεις των τοίχων, ούτε γνωρίζει πόσοι είναι οι τοίχοι στο περιβάλλον. Μαθαίνει για την ύπαρξη ενός τοίχου μονάχα κατόπιν σύγκρουσής με τον τοίχο, αλλά η σύγκρουση επιφέρει πολύ μεγάλο κόστος. Συνεπώς, είναι προτιμότερο (από άποψη κόστους) για τον πράκτορα να περιορίσει το ρίσκο σύγκρουσης με το να ακολουθήσει μια μακρύτερη διαδρομή που γνωρίζει ότι είναι όμως ασφαλής.

Καμιά φορά, ο πράκτορας κατά τον συλλογισμό του θα διαπιστώσει ότι κανένα από τα πλάνα του δεν είναι βιώσιμο, επειδή το αναμενόμενο κόστος υπερτερεί του αναμενόμενου κέρδους για οποιοδήποτε πλάνο εξετάζει ο πράκτορας. Αυτό μπορεί να οδηγήσει τον πράκτορα να μην εκτελέσει καμία δράση, ίσως ακόμη και για πάντα. Για τον λόγο αυτό, κάθε φορά που ο πράκτορας φθάσει σε σημείο που προτιμά να αναμένει από το να δράσει, μειώνει κατά ένα μικρό βαθμό την πιθανότητα σύγκρουσης που εκτιμά ότι υφίσταται. Η μείωση πρέπει να είναι σταδιακή, ώστε το ρίσκο να μειώνεται οργανικά, μέχρι να φθάσει στο σημείο όπου ο πράκτορας θα προβεί στην εκτέλεση κάποιου πλάνου, αλλά δεν θα μειώσει τόσο το ρίσκο ώστε να αγνοεί τον κίνδυνο της σύγκρουσης, για την οποιαδήποτε διαδρομή. Συμπεριφορά που θυμίζει φόβο, κατόπιν ενός βλαβερού συμβάντος, αλλά και θάρρους που χτίζεται με την πάροδο του χρόνου ή και την εξέλιξη μιας κατάστασης.

Στον ακόλουθο πίνακα φαίνονται οι μέσοι όροι παραμέτρων αξιολόγησης του πράκτορα για διάφορες τιμές μέγιστης αρχικής ενέργειας του ρομπότ (*MAX\_ROBOT\_ENERGY*).

| <i>Max Robot Energy</i>    | <b>60</b> | <b>120</b> | <b>240</b> |
|----------------------------|-----------|------------|------------|
| <i>Avg. Energy Used</i>    | 58        | 117.6      | 237        |
| <i>Avg. Clients Served</i> | 8.42      | 16.12      | 29.9       |
| <i>Avg. Collisions</i>     | 3.7       | 4          | 4.12       |
| <i>Avg. Evaluation</i>     | -256.27   | -201.42    | -51        |

Οι τιμές του παραπάνω πίνακα αναπαρίστανται και στο ακόλουθο διάγραμμα:



Παρατηρήστε πως ο πράκτορας έχει ένα αρκετά σταθερό πλήθος συγκρούσεων, και το πλήθος των πελατών που εξυπηρετούνται αυξάνονται με γραμμικό τρόπο, οπότε, και η εκτίμηση (*evaluation*) αυξάνεται γραμμικά με τη μέγιστη ενέργεια του ρομπότ.

## Εκτέλεση της Προσομοίωσης

Στον φάκελο του παραδοτέου θα βρείτε τον φάκελο `Code`. Μπορείτε να τρέξετε την προσομοίωση με χρήση του Jason CLI 3.2.0 από ένα Unix-based τερματικό, όπως το Git Bash. Για την ορθή λειτουργία της Jason 3.2.0 προτείνεται να έχετε εγκαταστημένη την Java 17.

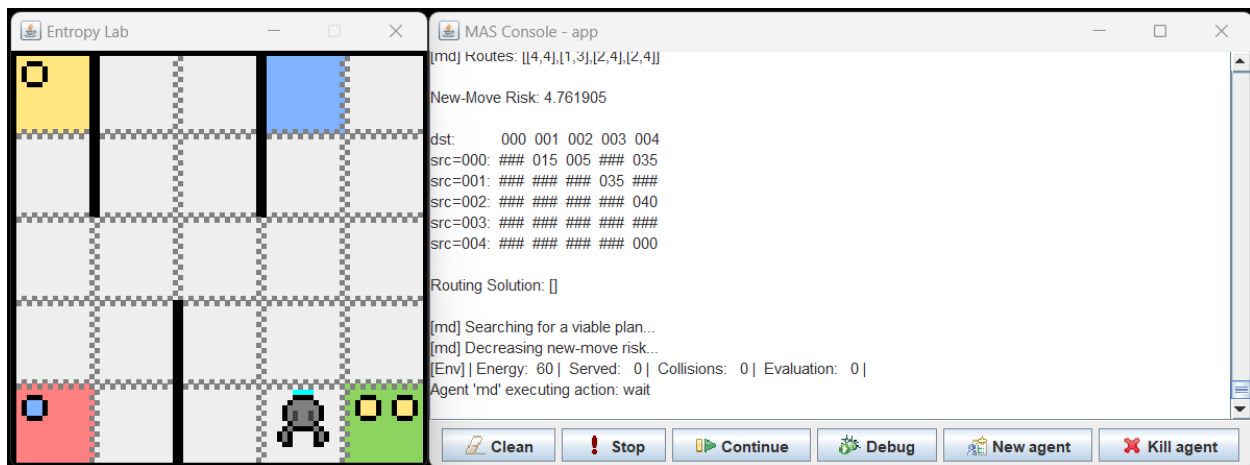
Αρχικά, θα ανοίξετε το τερματικό σας, και θα μεταβείτε στον κατάλογο του project `Code/app`:

```
cd Code/app/
```

Στη συνέχεια αρκεί να τρέξετε την ακόλουθη εντολή στο τερματικό σας:

```
jason app.mas2j
```

Κατόπιν της εκτέλεσης της παραπάνω εντολής, θα ανοίξει η κονσόλα “MAS Console – app” και το πάνελ της γραφικής απεικόνισης της προσομοίωσης, “Entropy Lab”, όπως παρουσιάζεται στην ακόλουθη εικόνα:



Ως προς το πάνελ της γραφικής απεικόνισης της προσομοίωσης, “Entropy Lab”, η ενέργεια του ρομπότ αναπαρίσταται ως μια μπάρα κυανού χρώματος ακριβώς πάνω από το εικονίδιο του. Οι πελάτες αναπαρίστανται ως μπλοκ με το χρώμα του προορισμού τους. Στο στιγμιότυπο προσομοίωσης στην παραπάνω εικόνα, στο κόκκινο κελί βρίσκεται ένας πελάτης με προορισμό το πράσινο κελί. Όταν το ρομπότ φορτώνεται με έναν πελάτη, το εικονίδιο του εν λόγω πελάτη επισυνάπτεται στο ρομπότ. Όταν το ρομπότ εξυπηρετήσει τον πελάτη που κρατούσε εκφορτώνοντάς τον στο κελί προορισμού του, το εικονίδιο του πελάτη αυτού εξαφανίζεται. Οι πελάτες εμφανίζονται τυχαία κατά τη διάρκεια της προσομοίωσης. Η σύγκρουση του ρομπότ με τοίχο αναπαρίσταται ως μια ταλάντωση του εικονιδίου του ρομπότ μεταξύ του τοίχου και του κέντρου του κελιού στο οποίο βρίσκεται το ρομπότ.

Στο παράθυρο “MAS Console – app” εκτυπώνονται πληροφορίες σχετικά με την προσομοίωση. Σε κάθε κύκλο αλληλεπίδρασης πράκτορα-περιβάλλοντος, εκτυπώνεται η ακόλουθη γραμμή από το *Env*, με την τρέχουσα ενέργεια του ρομπότ, το τρέχον πλήθος πελατών που έχουν εξυπηρετηθεί, το τρέχον πλήθος των συγκρούσεων και την τρέχουσα αξιολόγηση του πράκτορα, *evaluation*.

| Energy: 60 | Served: 0 | Collisions: 0 | Evaluation: 0 |

Επιπλέον, εκτυπώνεται η δράση που εκτελεί ο πράκτορας σε κάθε κύκλο.

Agent 'md' executing action: wait

Όποτε συμβεί γεγονός που επηρεάζει το *evaluation*, αυτό εκτυπώνεται, π.χ.:

ACTION (-1), Evaluation: -1  
COLLISION (-100), Evaluation: -101

Όποτε ο πράκτορας ενημερώνει την λίστα των διαδρομών πελατών στην βάση γνώσης του, είτε λόγω προσθήκης είτε λόγω διαγραφής κάποιας διαδρομής, εκτυπώνεται από τον πράκτορα η λίστα:

Routes: [[1,2],[4,2]]

Στο παραπάνω παράδειγμα, ο πράκτορας πιστεύει ότι υπάρχουν δύο ενεργές διαδρομές πελατών για εξυπηρέτηση, ενός πελάτη που βρίσκεται στο κόκκινο κελί (χρώμα 1) με προορισμό το πράσινο κελί (χρώμα 2), και ενός πελάτη με αφετηρία το κίτρινο κελί (4) και προορισμό το πράσινο (2) κελί.

Επιπλέον, ο σχεδιαστής δράσης του πράκτορα εκτυπώνει το τρέχον εκτιμώμενο ρίσκο νέας μετακίνησης καθώς και έναν πίνακα γειτνίασης για μεταβάσεις μέσω διαδρομών πελατών μεταξύ των κομβικών κελιών, δηλαδή το κελί στο οποίο βρίσκεται το ρομπότ (0) και τα χρωματιστά κελιά, κόκκινο (1), πράσινο (2), μπλε (3), κίτρινο (4). Σε κάθε θέση του πίνακα αν υπάρχει διαδρομή πελάτη από το χρωματιστό κελί *src* στο χρωματιστό κελί *dst*, το στοιχείο του πίνακα στη θέση [*src*, *dst*] περιέχει το αναμενόμενο κόστος καθοδήγησης από το ένα κελί στο άλλο, βάσει του εκτιμώμενου ρίσκου νέας μετακίνησης. Ένα παράδειγμα εκτύπωσης του ρίσκου και του πίνακα γειτνίασης φαίνεται παρακάτω:

New-Move Risk: 25.000000

```
dst:      000  001  002  003  004
src=000:  ###  052  ###  ###  156
src=001:  ###  ###  104  ###  ###
src=002:  ###  ###  ###  ###  ###
src=003:  ###  ###  ###  ###  ###
src=004:  ###  ###  208  ###  ###
```

Αν το ρίσκο είναι αρκετά μεγάλο, ο σχεδιαστής πλάνου του πράκτορα συνάγει ότι δεν αξίζει να ληφθεί δράση. Τότε ο πράκτορας πραγματοποιεί την δράση αναμονής, μειώνοντας λίγο το ρίσκο, και εκτυπώνεται το ακόλουθο μήνυμα από τον πράκτορα:

Searching for a viable plan...  
Decreasing new-move risk...

Η αναμονή λόγω του ρίσκου μπορεί να κρατήσει αρκετούς κύκλους, αλλά σε κάποιο στιγμιότυπο είτε ο πράκτορας θα αντιληφθεί ότι το περιβάλλον είναι σε τέτοια κατάσταση που το κέρδος κάποιου σχεδίου υπερτερεί του συνολικού ρίσκου, είτε το ρίσκο θα πέσει επαρκώς ώστε ο πράκτορας να προβεί σε δράση. Τότε ο πράκτορας εκτυπώνει το δρομολόγιο και το συνολικό πλάνο δράσης του και προβαίνει στην εκτέλεση του.

Routing Solution: [3, 4, 4, 3, 4]  
Plan: [1, 2, 8, 2, 2, 2, 4, 8, 4, 7, 0, 0, 0, 4, 8, 2, 2, 4]

Οι αριθμοί του Routing Solution αποτελούν την ακολουθία των χρωματιστών κελιών που θα επισκεφθεί ο πράκτορας με τη σειρά επίσκεψης τους. Ενώ οι αριθμοί του πλήρους πλάνου, Plan, ερμηνεύονται σύμφωνα με τον πίνακα:

| Αριθμητική αναπαράσταση<br>δράσης, <i>A</i> | Δράση          | Περιγραφή                  |
|---------------------------------------------|----------------|----------------------------|
| <i>A</i> = 0, 1, 2, 3                       | <i>move(A)</i> | Μετακίνηση προς <i>A</i> . |
| <i>A</i> = 4                                | <i>unload</i>  | Εκφόρτωση πελάτη.          |

|                  |               |                            |
|------------------|---------------|----------------------------|
| $A = 5, 6, 7, 8$ | $load(A - 4)$ | Φόρτωση πελάτη ( $A - 4$ ) |
| 9                | <i>wait</i>   | —                          |

Οι αριθμοί 0, 1, 2 και 3 αντιστοιχούν σε μετακίνηση κατά ένα κελί προς τα δεξιά, επάνω, αριστερά και κάτω αντίστοιχα. Οι αριθμοί 5, 6, 7 και 8 αντιστοιχούν στην φόρτωση κόκκινου, πράσινου, μπλε και κίτρινου πελάτη αντίστοιχα από το τρέχον κελί του ρομπότ.

## Βιβλιογραφία

- [1] Artificial Intelligence a Modern Approach Third Edition, Stuard Rusel and Peter Norvig
- [2] An Introduction to MultiAgent Systems, Michael Wooldridge
- [3] Programming multi-agent systems in AgentSpeak using Jason, Bordini, Hubner, Wooldirdge